

# Transaction & Fraud Analysis

Data Analyst Project | Python • SQL • Power BI

## Project Report

This project analyzes 250,000 financial transactions across customers, cards, merchants, and transaction activity to understand transaction performance, customer behavior, payment channels, card usage, merchant performance, and fraud risk. Python was used for initial data quality checks and cleaning, SQL was used for analytical querying, and Power BI was used to build an interactive reporting layer.

|                          |         |
|--------------------------|---------|
| Total Transactions       | 250K    |
| Total Customers          | 25K     |
| Total Active Cards       | 30.46K  |
| Total Transaction Amount | ₹4.15bn |
| Successful Transactions  | 91.13%  |
| Fraud Rate               | 5.39%   |
| Avg. Transaction Value   | ₹16.61K |

# 1. Project Overview

The objective is to transform raw card and transaction data into actionable business insights. The analysis focuses on four major dimensions: customer behavior, card usage, merchant performance, and fraud risk, with a time-based view of transaction activity.

## Business Questions

- Which customers generate the highest transaction value and transaction frequency?
- Which age group, card type, and card network contribute the most transaction activity?
- Which merchants, categories, cities, and channels drive the highest transaction value?
- What is the overall fraud rate and which merchant categories, payment methods, months, and channels carry higher fraud risk?
- How does transaction value change over time?
- Which customers are dormant, highly valuable, or active across consecutive months?

## Technology Stack

Python (Pandas) • MySQL/SQL • Power BI • Data Cleaning • Exploratory Analysis • Window Functions • CTEs • Dashboarding

## 2. Data & Data Preparation

The project uses four related datasets representing customers, cards, merchants, and transactions. The transaction file contains approximately 250,000 records.

| Dataset      | Purpose                                                 | Key field      | Approx. role       |
|--------------|---------------------------------------------------------|----------------|--------------------|
| Customers    | Customer demographics and profile                       | customer_id    | Customer dimension |
| Cards        | Card type, network, status and limits                   | card_id        | Card dimension     |
| Merchants    | Merchant name, category, city/state                     | merchant_id    | Merchant dimension |
| Transactions | Transaction amount, date, status, fraud, payment method | transaction_id | Fact table         |

### Python Data Cleaning Workflow

- Loaded the four CSV files using Pandas.
- Checked missing values in each dataset.
- Filled missing Fraud\_Reason values with 'No Fraud'.
- Checked duplicate records and duplicate primary-key values.
- Standardized text columns using lowercase and strip operations.
- Standardized column names to lowercase.
- Reviewed summary statistics and data types.
- Exported cleaned datasets for downstream SQL/BI analysis.

### Data Quality Checks Performed

The Python script demonstrates a practical first-pass data-quality process: null checks, duplicate checks, primary-key validation, text normalization, schema/type inspection, and clean-file export.

### 3. SQL Analysis

SQL was used as the main analytical layer after cleaning. The query set covers customer, card, merchant, fraud, time-based, and advanced analytical questions.

#### Customer Analysis

- Top 10 customers by successful transaction amount

```
WITH customer_rank AS
(SELECT c.customer_id,c.customer_name,
       sum(t.transaction_amount) AS total_transaction_amount,
       DENSE_RANK()OVER(ORDER BY sum(t.transaction_amount) DESC) AS dr
FROM customers c
JOIN transactions t
ON c.customer_id = t.customer_id
WHERE t.transaction_status = 'successful'
GROUP BY c.customer_id, c.customer_name)
SELECT customer_id, customer_name, total_transaction_amount
FROM customer_rank
WHERE dr <= 10;
```

| customer_name     | total_transaction_amount |
|-------------------|--------------------------|
| ekaraj purohit    | 2532030.74               |
| garima mangal     | 2485468.40               |
| aarini sen        | 2282569.23               |
| abha sahota       | 2154656.34               |
| samaksh maharaj   | 2085405.86               |
| kavya sridhar     | 2051755.87               |
| nikita badami     | 2002255.40               |
| maanav kannan     | 1987803.81               |
| jagat halder      | 1982510.95               |
| abeer subramaniam | 1978158.21               |

**Insight:** The top customer generated approximately ₹2.53M in successful transaction amount, highlighting the contribution of high-value customers.

- Customer with the highest successful transaction count

```
WITH customer_rank AS
(SELECT c.customer_id,c.customer_name,
       count(t.transaction_id) AS total_transaction,
       ROW_NUMBER()OVER(ORDER BY count(t.transaction_id) DESC) AS dr
FROM customers c
JOIN transactions t
ON c.customer_id = t.customer_id
WHERE t.transaction_status = 'successful'
GROUP BY c.customer_id, c.customer_name)
SELECT customer_id, customer_name, total_transaction
FROM customer_rank
WHERE dr = 1;
```

| Result Grid | Filter Rows:  | Export            |
|-------------|---------------|-------------------|
| customer_id | customer_name | total_transaction |
| cust121815  | gayathri dave | 37                |

**Insight:** The top customer completed 37 successful transactions.

- Highest-value age group

```
SELECT
  case
    WHEN c.age BETWEEN 18 AND 25 THEN '18-25'
    WHEN c.age BETWEEN 26 AND 35 THEN '26-35'
    WHEN c.age BETWEEN 36 AND 45 THEN '36-45'
    WHEN c.age BETWEEN 46 AND 60 THEN '46-60'
    ELSE '60+'
  END as age_group,
  SUM(t.transaction_amount) AS total_transaction_amount
FROM
  customers c
  JOIN
  transactions t ON c.customer_id = t.customer_id
  WHERE t.transaction_status = 'successful'
GROUP BY age_group
ORDER BY total_transaction_amount DESC
LIMIT 1;
```

| Result Grid |           |                          | Filter Rows: |
|-------------|-----------|--------------------------|--------------|
|             | age_group | total_transaction_amount |              |
| ▶           | 46-60     | 1039127988.85            |              |

**Insight:** The 46–60 age group generated the highest transaction value.

- Transaction amount comparison by gender

```
SELECT
  c.gender,
  SUM(transaction_amount) AS total_transaction_amount
FROM
  customers c
  JOIN
  transactions t ON c.customer_id = t.customer_id
  WHERE t.transaction_status = 'successful'
GROUP BY c.gender
ORDER BY total_transaction_amount DESC;
```

| gender | total_transaction_amount |
|--------|--------------------------|
| female | 1676698065.84            |
| male   | 1638141308.97            |

**Insight:** Female customers generated slightly higher transaction value than male customers in the displayed result.

- Customers with more than one active card

```
SELECT
    c.customer_id,
    c.customer_name,
    COUNT(cd.card_status) AS total_active_card
FROM
    customers c
    JOIN
    cards cd ON c.customer_id = cd.customer_id
WHERE
    cd.card_status = 'active'
GROUP BY c.customer_id , c.customer_name
HAVING COUNT(cd.card_status) > 1;
```

| customer_id | customer_name       | total_active_card |
|-------------|---------------------|-------------------|
| cust100005  | gautam mukhopadhyay | 2                 |
| cust100009  | samarth raman       | 2                 |
| cust100011  | bakhshi bose        | 2                 |
| cust100012  | tejas ratti         | 2                 |
| cust100026  | saanvi ganesh       | 3                 |

**Insight:** The result identifies customers with multiple active cards; one displayed customer has 3 active cards, while the others have 2.

## Card Analysis

- Highest-value card type

```
SELECT
    cd.card_type,
    SUM(t.transaction_amount) AS total_amount_per_card
FROM
    cards cd
    JOIN
    transactions t ON cd.card_id = t.card_id
WHERE
    t.transaction_status = 'successful'
GROUP BY cd.card_type
ORDER BY total_amount_per_card DESC
LIMIT 1;
```

| card_type | total_amount_per_card |
|-----------|-----------------------|
| gold      | 1029549447.29         |

**Insight:** Gold cards generated the highest successful transaction amount, at approximately ₹1.03bn.

- Most-used card network

```
SELECT cd.card_network,
    COUNT(*) AS total_transactions
FROM cards cd
JOIN transactions t
ON cd.card_id = t.card_id
WHERE t.transaction_status = 'successful'
GROUP BY cd.card_network
ORDER BY total_transactions DESC
LIMIT 1;
```

| card_network     | total_transactions |
|------------------|--------------------|
| american express | 58158              |

**Insight:** American Express is the most-used card network in the displayed result, with 58,158 successful transactions.

- Customers spending more than 80% of card limit

```
SELECT
    c.customer_name,
    cd.card_id,
    cd.card_limit,
    SUM(t.transaction_amount) AS total_spend,
    round((SUM(t.transaction_amount) / cd.card_limit) * 100 ,2) AS spend_percent
FROM
    customers c
JOIN
    cards cd ON c.customer_id = cd.customer_id
JOIN
    transactions t ON cd.card_id = t.card_id
WHERE t.transaction_status = 'successful'
GROUP BY
    c.customer_id,
    c.customer_name,
    cd.card_id,
    cd.card_limit
HAVING
    total_spend > (0.80 * cd.card_limit);
```

| customer_name  | card_id    | card_limit | total_spend | spend_percent |
|----------------|------------|------------|-------------|---------------|
| jagrati virk   | card522230 | 66295      | 229907.27   | 346.79        |
| saumya kadakia | card531341 | 71901      | 336700.85   | 468.28        |
| ekani ravel    | card505308 | 51015      | 251136.87   | 492.28        |
| girish sodhi   | card513970 | 234138     | 224105.74   | 95.72         |

**Insight:** Four customers are shown spending more than 80% of their card limit; some have cumulative spending well above the stated card limit.

## Merchant Analysis

- Top 10 merchants by successful transaction amount

```
SELECT m.merchant_name,
    sum(t.transaction_amount) AS total_revenue
FROM merchants m
JOIN transactions t
ON m.merchant_id = t.merchant_id
WHERE t.transaction_status = 'successful'
GROUP BY m.merchant_id, m.merchant_name
ORDER BY total_revenue DESC
LIMIT 10;
```

| Result Grid |                   | Filter Rows:            |
|-------------|-------------------|-------------------------|
|             | merchant_name     | total_transaction_value |
| ▶           | apollo hospitals  | 180427526.78            |
|             | spicejet          | 156360539.80            |
|             | indigo            | 151061568.65            |
|             | samsung smartcafe | 147947539.25            |
|             | max healthcare    | 132155244.31            |
|             | manipal hospitals | 132049627.45            |
|             | coursera          | 123292851.11            |
|             | air india         | 113068420.58            |
|             | makemytrip hotels | 106999579.94            |
|             | cashfree          | 100873876.33            |

**Insight:** Apollo is the top merchant in the displayed result, with approximately ₹180M in successful transaction amount.



- Highest-value merchant category

```
SELECT m.merchant_category,
       SUM(t.transaction_amount) as total_revenue
FROM merchants m
JOIN transactions t
ON m.merchant_id = t.merchant_id
WHERE t.transaction_status = 'successful'
GROUP BY m.merchant_category
ORDER BY total_revenue DESC
LIMIT 1;
```

| merchant_category | total_revenue |
|-------------------|---------------|
| hospital          | 532253115.20  |

**Insight:** Hospital is the highest-value merchant category in the displayed result, with approximately ₹532.25M in successful transaction amount.

## Fraud Analysis

- Overall fraud rate

```
SELECT AVG(fraud_flag) * 100 AS overall_fraud_rate
FROM transactions;
```

| overall_fraud_rate |
|--------------------|
| 5.3892             |

**Insight:** The overall fraud rate is 5.3892%, meaning about 5.4% of transactions are flagged as fraudulent.

- Merchant category with highest fraud rate

```
SELECT m.merchant_category,
       ROUND((SUM(t.fraud_flag) * 100.0) / COUNT(*),1) as fraud_rate_per_category
FROM merchants m
JOIN transactions t
ON m.merchant_id = t.merchant_id
GROUP BY m.merchant_category
ORDER BY fraud_rate_per_category DESC
LIMIT 1;
```

| merchant_category | fraud_rate_per_category |
|-------------------|-------------------------|
| entertainment     | 5.7                     |

**Insight:** Entertainment has the highest displayed merchant-category fraud rate at 5.7%.

- Payment method with highest fraud percentage

```
SELECT payment_method,
       ROUND((SUM(fraud_flag) * 100) / COUNT(*),1) as fraud_percentage
FROM transactions
GROUP BY payment_method
ORDER BY fraud_percentage DESC
LIMIT 1;
```

| payment_method | fraud_percentage |
|----------------|------------------|
| credit card    | 5.5              |

**Insight:** Credit card has the highest displayed fraud percentage among payment methods at 5.5%.



## Time Analysis

- Monthly transaction trend

```
SELECT
    DATE_FORMAT(transaction_date, '%Y-%M') AS month,
    SUM(transaction_amount) AS total_amount
FROM
    transactions
WHERE
    transaction_status = 'successful'
GROUP BY DATE_FORMAT(transaction_date, '%Y-%M') ,
    YEAR(transaction_date) , MONTH(transaction_date)
ORDER BY YEAR(transaction_date) , MONTH(transaction_date);
```

| month         | total_amount |
|---------------|--------------|
| 2023-January  | 83371669.95  |
| 2023-February | 77171934.54  |
| 2023-March    | 80894871.07  |
| 2023-April    | 83144897.22  |
| 2023-May      | 84829463.93  |

**Insight:** Among the months displayed in the SQL result, June has the highest successful transaction amount at approximately ₹371.95M.

- Peak transaction hour

```
SELECT
    HOUR(transaction_date) AS peak_hour,
    SUM(transaction_amount) AS total_amount
FROM
    transactions
WHERE
    transaction_status = 'successful'
GROUP BY HOUR(transaction_date)
ORDER BY total_amount DESC
LIMIT 1;
```

| peak_hour | total_amount  |
|-----------|---------------|
| 0         | 3314839374.81 |

**Insight:** 12 AM is the peak transaction hour in the displayed result, with approximately ₹331.48M in successful transaction amount.

- Weekday vs weekend transaction volume

```
SELECT
    CASE
        WHEN DAYNAME(transaction_date) IN ('saturday' , 'sunday') THEN 'weekend'
        ELSE 'weekdays'
    END AS days_type,
    COUNT(*) AS total_transactions
FROM
    transactions
WHERE
    transaction_status = 'successful'
GROUP BY days_type;
```

| days_type | total_transactions |
|-----------|--------------------|
| weekend   | 65380              |
| weekdays  | 162441             |

**Insight:** Weekday transactions are much higher than weekend transactions, with 162,441 weekday transactions versus 65,380 weekend transactions.

## Advanced SQL

- Top 5 customers in each state using DENSE\_RANK

```
WITH customer_rnk AS
(SELECT c.customer_name,c.state as customer_state,
    SUM(t.transaction_amount) AS total_amount,
    DENSE_RANK() OVER(PARTITION BY c.state ORDER BY SUM(t.transaction_amount) DESC) AS dr
FROM customers c
JOIN transactions t
ON c.customer_id = t.customer_id
GROUP BY c.customer_name,c.state)
SELECT customer_state, customer_name,total_amount from customer_rnk
WHERE dr <= 5;
```

| customer_state | customer_name | total_amount |
|----------------|---------------|--------------|
| assam          | abha sahota   | 2154656.34   |
| assam          | kamya boase   | 1946844.93   |
| assam          | warhi ghosh   | 1897571.47   |
| assam          | barkha bedi   | 1642611.65   |
| assam          | unni borra    | 1614470.37   |

**Insight:** In the displayed result, Assam contains the returned top customers, with Abha Sahota having the highest transaction amount among them at approximately ₹21.55M.

- First and latest transaction using ROW\_NUMBER

```
WITH cte AS
(SELECT customer_id, transaction_id, transaction_date, transaction_amount,
ROW_NUMBER() over(PARTITION BY customer_id ORDER BY transaction_date DESC) AS latest_date,
ROW_NUMBER() OVER(PARTITION BY customer_id ORDER BY transaction_date ASC) AS first_date
FROM transactions)
SELECT DISTINCT customer_id, transaction_id, first_date, latest_date, transaction_amount,
CASE WHEN first_date = 1 THEN 'first_transaction'
      WHEN latest_date = 1 THEN 'latest_tansaction'
END AS transaction_type
FROM cte
WHERE latest_date = 1 OR first_date = 1;
```

| customer_id | transaction_id | first_date | latest_date | transaction_amount | transaction_type  |
|-------------|----------------|------------|-------------|--------------------|-------------------|
| cust100001  | txn1222676     | 1          | 7           | 42407.72           | first_transaction |
| cust100001  | txn1202848     | 7          | 1           | 10218.25           | latest_tansaction |
| cust100002  | txn1113225     | 1          | 10          | 35130.87           | first_transaction |
| cust100002  | txn1002128     | 10         | 1           | 59266.37           | latest_tansaction |

**Insight:** The query successfully identifies first and latest transactions for each customer; for example, cust100001 has a first transaction of ₹42,407.72 and a latest transaction of ₹10,218.25.

- Dormant customers (no transactions in the last 90 days)

```
SELECT
    c.customer_id, c.customer_name
FROM
    customers c
    LEFT JOIN
    transactions t ON c.customer_id = t.customer_id
GROUP BY c.customer_id , c.customer_name
HAVING MAX(t.transaction_date) < CURRENT_DATE() - INTERVAL 90 DAY
      OR MAX(t.transaction_date) IS NULL;
```

| customer_id | customer_name  |
|-------------|----------------|
| cust100001  | omaja majumdar |
| cust100002  | jason agrawal  |
| cust100004  | radhika batra  |

**Insight:** Three customers are returned as dormant under the query's current 90-day inactivity condition.

- Customers transacting in consecutive months using LAG

```
WITH customer_months AS (
    SELECT DISTINCT
        customer_id,
        YEAR(transaction_date) AS yr,
        MONTH(transaction_date) AS mon
    FROM transactions
    WHERE transaction_status = 'successful'
),
cte AS (
    SELECT *,
        LAG(yr) OVER(PARTITION BY customer_id ORDER BY yr, mon) AS prev_year,
        LAG(mon) OVER(PARTITION BY customer_id ORDER BY yr, mon) AS prev_month
    FROM customer_months
)
SELECT DISTINCT customer_id
FROM cte
WHERE (yr = prev_year AND mon = prev_month + 1)
    OR (yr = prev_year + 1 AND prev_month = 12 AND mon = 1);
```

| customer_id |
|-------------|
| cust100002  |
| cust100003  |
| cust100005  |
| cust100006  |
| cust100009  |

**Insight:** Five customers are returned as having transactions in consecutive months, indicating repeated activity across adjacent months.

- RFM base metrics

```
WITH rfm AS (  
    SELECT  
        c.customer_id,  
        c.customer_name,  
        DATEDIFF(  
            CURRENT_DATE(),  
            MAX(t.transaction_date)  
        ) AS recency,  
        COUNT(t.transaction_id) AS frequency,  
        SUM(t.transaction_amount) AS monetary  
    FROM customers c  
    JOIN transactions t  
        ON c.customer_id = t.customer_id  
    WHERE t.transaction_status = 'successful'  
    GROUP BY  
        c.customer_id,  
        c.customer_name)  
SELECT *  
FROM rfm  
ORDER BY monetary DESC;
```

| customer_id | customer_name  | recency | frequency | monetary   |
|-------------|----------------|---------|-----------|------------|
| cust115968  | ekaraj purohit | 42      | 23        | 2532030.74 |
| cust121300  | garima mangal  | 43      | 22        | 2485468.40 |
| cust116838  | aarini sen     | 73      | 11        | 2282569.23 |

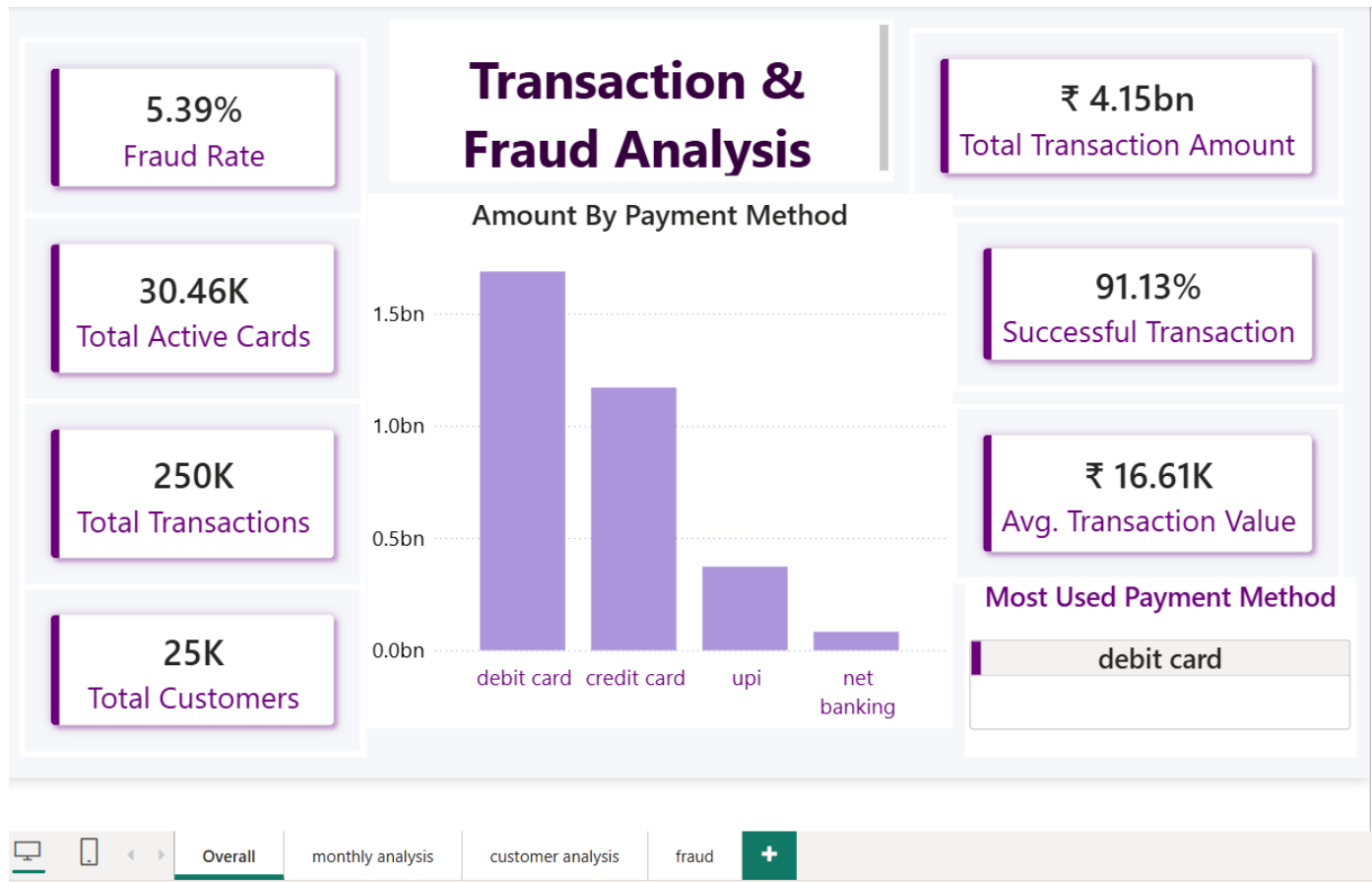
**Insight:** The RFM base metrics show different customer-value patterns; for example, cust115968 has 23 transactions and approximately ₹25.32M in monetary value, while cust116838 has 11 transactions and approximately ₹22.83M.

## 4. Power BI Dashboard

The Power BI report is organized into four pages: Overall, Monthly Analysis, Customer Analysis, and Fraud. The pages combine KPI cards, bar charts, trend charts, tables, and slicers to support both high-level monitoring and deeper investigation.

### Overall / Overview

Purpose: provide a quick executive view of transaction scale, fraud, active cards, successful transactions, average transaction value, and payment-method contribution.

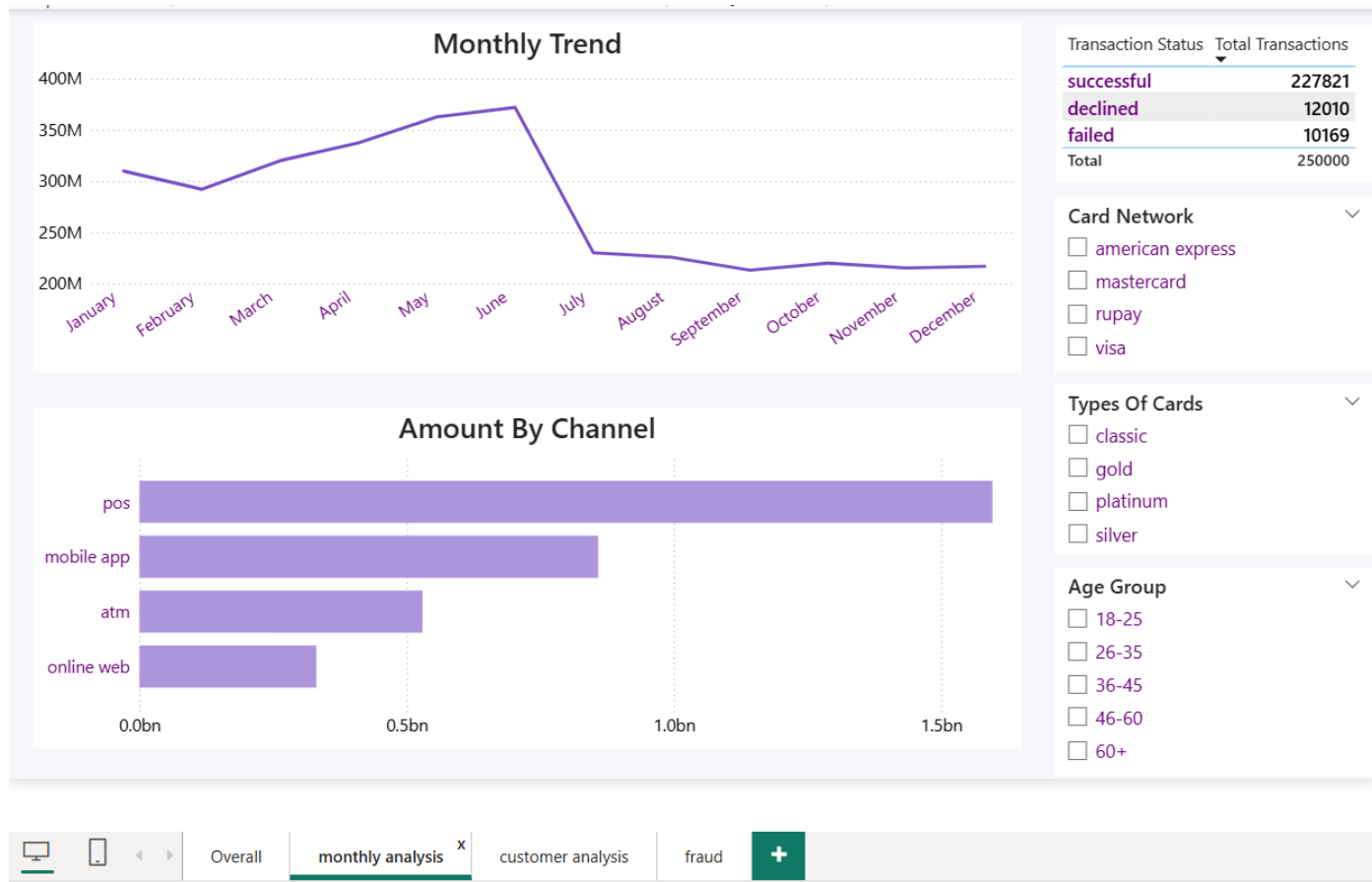


### Key observations

- Fraud rate: 5.39%.
- Total transaction amount: ₹4.15bn.
- Total active cards: 30.46K.
- Total transactions: 250K.
- Total customers: 25K.
- Successful transactions: 91.13%.
- Average transaction value: ₹16.61K.
- Debit card is the highest-value payment method shown.

## Monthly Analysis

Purpose: identify seasonality, changes in transaction value, channel contribution, and transaction-status mix.



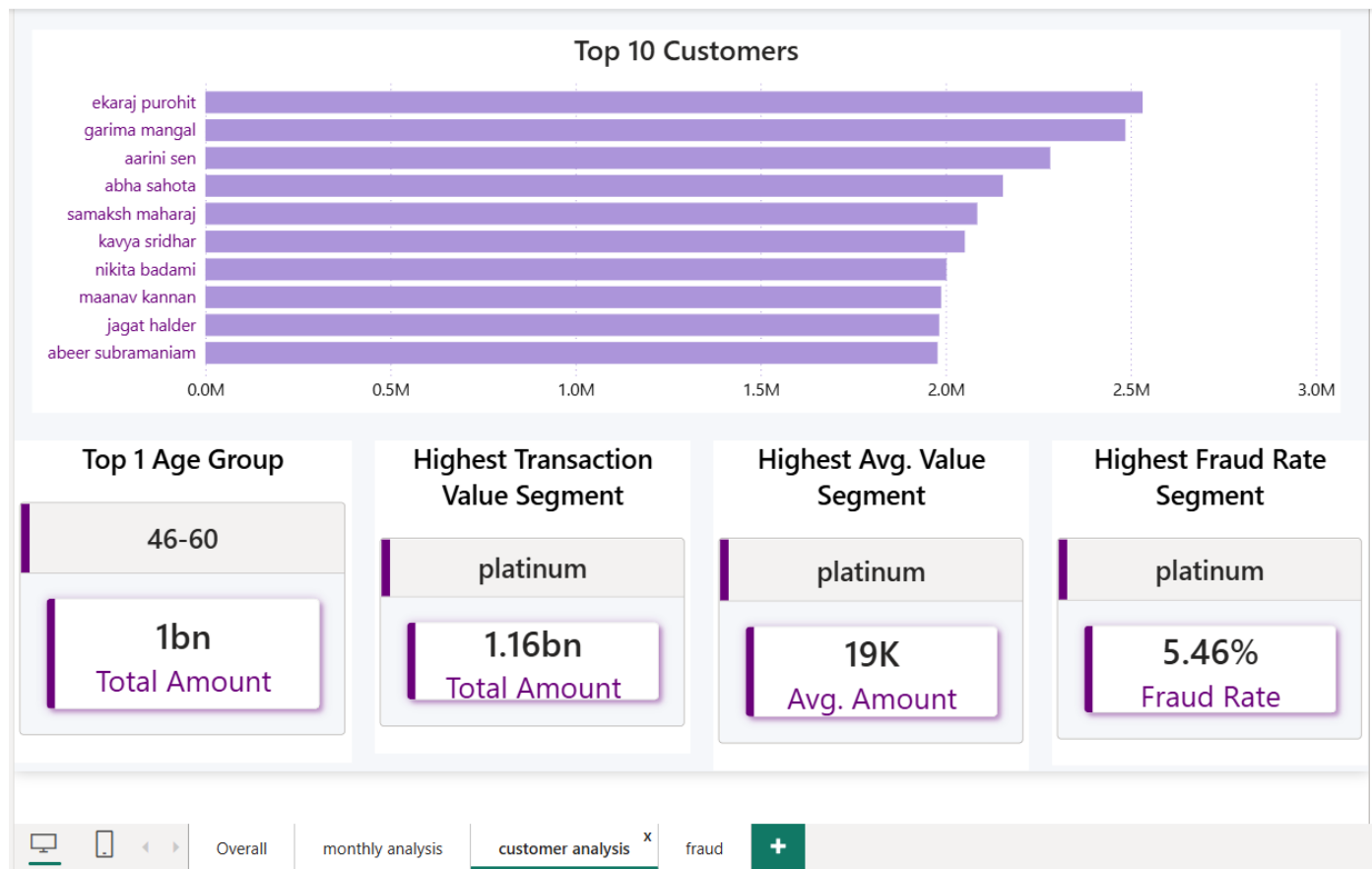
### Key observations

- Transaction value peaks around June at approximately ₹371.95M.
- A pronounced decline occurs from June to July.
- POS is the largest channel by transaction amount, followed by mobile app, ATM, and online web.
- Successful transactions dominate the status table with 227,821 transactions.
- Declined and failed transactions total 12,010 and 10,169 respectively.



## Customer Analysis

Purpose: understand customer concentration, high-value segments, and demographic contribution.

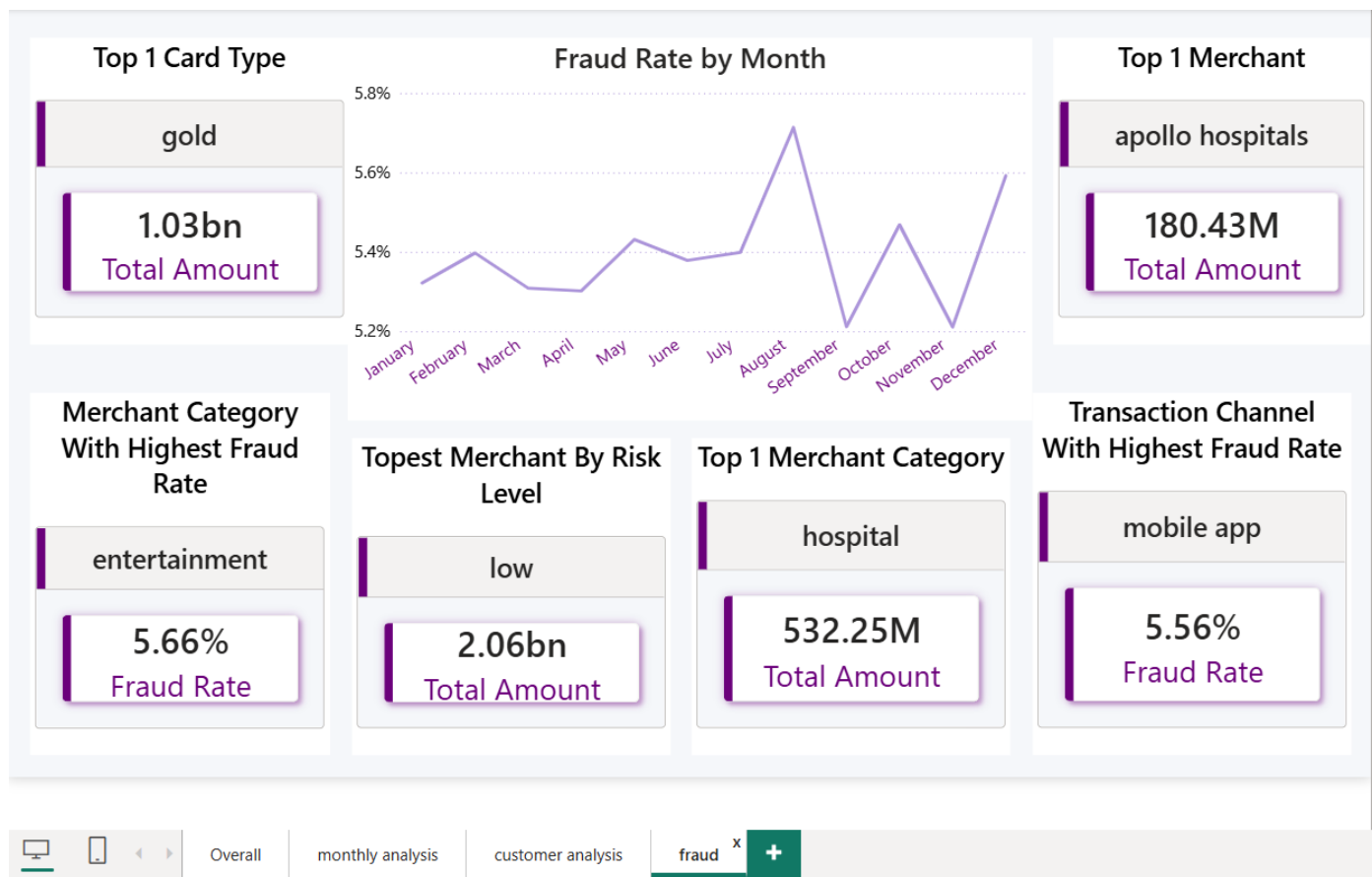


### Key observations

- The dashboard ranks the top 10 customers by transaction value.
- The 46-60 age group is the highest-value age group shown.
- Platinum is the top customer segment by total amount, at about ₹1.16bn.
- Platinum also shows an average transaction amount of about ₹19K.
- The displayed platinum fraud rate is 5.46%.

## Fraud Analysis

Purpose: identify where fraud risk is concentrated across time, merchant category, merchant, and transaction channel.



### Key observations

- Fraud rate peaks around August at approximately 5.7%.
- Entertainment has the highest displayed merchant-category fraud rate at 5.66%.
- Mobile app has the highest displayed transaction-channel fraud rate at 5.56%.
- Hospital is the top merchant category by transaction amount shown, at about ₹532.25M.
- Apollo Hospitals is the top merchant shown by transaction amount, at about ₹180.43M.
- Gold is the top card type shown by transaction amount, at about ₹1.03bn.

## 5. Consolidated Business Insights

**Overall performance:** The dataset contains 250K transactions with ₹4.15bn in transaction value. Successful transactions account for 91.13%, while the overall fraud rate is 5.39%.

**Payment method:** Debit card contributes the highest transaction amount among the payment methods shown, making it the dominant payment method by value.

**Transaction channel:** POS is the largest transaction channel by amount, followed by mobile app, ATM, and online web.

**Monthly trend:** Transaction value reaches its highest point around June at roughly ₹371.95M, followed by a sharp decline in July. The later months remain comparatively lower and stable.

**Customer concentration:** The Top 10 Customers visual shows a small group of customers generating substantially higher transaction value than the rest of the customer base.

**Customer segment:** Platinum is the leading customer segment by transaction amount in the dashboard, with approximately ₹1.16bn shown.

**Age group:** The 46-60 age group is the highest-contributing age group by transaction value in the customer analysis page.

**Fraud trend:** Fraud rate varies by month, peaking around August at approximately 5.7% and reaching around 5.2% in September and November.

**Merchant/category risk:** Entertainment is shown as the merchant category with the highest fraud rate at 5.66%.

**Channel risk:** Mobile app is shown as the transaction channel with the highest fraud rate at 5.56%.

**Merchant performance:** Apollo Hospitals is the top merchant shown on the fraud analysis page by transaction amount, at about ₹180.43M.

## 6. Business Recommendations

**Monitor fraud-heavy channels:** Mobile app and entertainment show relatively high fraud rates in the dashboard. These segments should receive stronger authentication, device/risk scoring, velocity checks, and transaction monitoring.

**Investigate the June-to-July drop:** The sharp decline in monthly transaction value warrants investigation into customer activity, payment-channel behavior, merchant activity, operational issues, or seasonal effects.

**Protect high-value customers:** The concentration visible in the Top 10 customers suggests that a relatively small group contributes significant value. Retention campaigns and proactive service can help protect this revenue base.

**Prioritize high-value segments:** Platinum customers contribute the highest transaction value in the dashboard. The business can design targeted loyalty, cross-sell, and retention strategies for this segment.

**Optimize channel strategy:** POS is the largest channel by transaction amount. The business should maintain POS reliability while assessing opportunities to grow digital channels without increasing fraud exposure.

**Use risk-adjusted performance:** Merchant and category performance should be evaluated together with fraud rate, not transaction amount alone. A high-value category with elevated fraud may require additional controls before expansion.

## 7. Analyst Methodology

1. Understand the business questions and define relevant KPIs.
2. Inspect raw data for missing values, duplicates, inconsistent text, and schema issues.
3. Clean and standardize data using Python/Pandas.
4. Load cleaned data into SQL and validate relationships through keys.
5. Use SQL aggregations, joins, CTEs, and window functions to answer analytical questions.
6. Build Power BI measures and visuals for management-friendly reporting.
7. Translate visual results into business insights rather than merely describing chart shapes.
8. Convert insights into practical recommendations and risk-monitoring actions.

## 8. Project Explanation

“I worked on a credit-card transaction and fraud analysis project using four datasets: customers, cards, merchants, and transactions. I first used Python and Pandas to inspect missing values, duplicates, data types, and text consistency, then created cleaned files. I used SQL to analyze customer value, card usage, merchant performance, transaction trends, and fraud patterns, including CTEs and window functions such as DENSE\_RANK, ROW\_NUMBER, and LAG. Finally, I built a Power BI dashboard with overall KPIs, monthly trends, customer analysis, and fraud analysis. Some key findings were a 5.39% overall fraud rate, 91.13% successful transactions, POS as the largest transaction channel by value, and higher fraud rates in the entertainment category and mobile-app channel. The main goal was to turn raw transaction data into business-focused insights and recommendations.”

## 9. Technical Notes & Improvements

**CLV definition:** The current CLV query calculates cumulative historical successful transaction value per customer. For a formal Customer Lifetime Value model, retention, lifespan, margin, or a future-value assumption should also be incorporated.

**RFM definition:** The current RFM query correctly calculates Recency, Frequency, and Monetary values, but it does not yet assign RFM scores or customer segments.

**Dormant customers:** The dormant-customer query uses CURRENT\_DATE(). If the dataset is historical and does not extend close to the current date, almost all customers may appear dormant. A dataset-relative reference date (for example, MAX(transaction\_date)) is usually more appropriate.

**Fraud rate:** The overall fraud rate is calculated across all transactions. This is appropriate for an overall fraud-rate KPI; segment-level fraud rates should use the same numerator/denominator logic consistently.

**Revenue terminology:** Because the analysis uses transaction\_amount, 'transaction value' or 'transaction amount' is more precise than 'revenue' unless merchant revenue is explicitly defined in the dataset.

**First/latest transaction:** The window-function approach identifies first and latest transactions per customer. If a customer has only one transaction, the CASE expression labels it as first rather than both first and latest.

These points do not invalidate the project; they are refinements that make the analysis more technically defensible during an interview.

# Python Data Cleaning Code

```
import pandas as pd

cards = pd.read_csv('data/cards_data.csv')
customers = pd.read_csv('data/customer_data.csv')
merchants = pd.read_csv('data/merchant_table.csv')
transactions = pd.read_csv('data/transaction_data_250k.csv')

# Missing-value checks
print(cards.isnull().sum())
print(customers.isnull().sum())
print(merchants.isnull().sum())

transactions["Fraud_Reason"] = transactions["Fraud_Reason"].fillna("No Fraud")
print(transactions.isnull().sum())

# Duplicate checks
print(transactions["Transaction_ID"].duplicated().sum())
print(cards.duplicated().sum())
print(customers.duplicated().sum())
print(merchants.duplicated().sum())

# Standardize text columns
for df in [customers, cards, merchants, transactions]:
    for col in df.select_dtypes(include="object").columns:
        df[col] = df[col].str.lower().str.strip()

# Standardize column names
for df in [customers, cards, merchants, transactions]:
    df.columns = df.columns.str.lower()

# Summary statistics and data types
print(transactions.describe())
print(transactions.dtypes)

# Primary-key duplicate checks
print(customers["customer_id"].duplicated().sum())
print(cards["card_id"].duplicated().sum())
print(merchants["merchant_id"].duplicated().sum())
print(transactions["transaction_id"].duplicated().sum())

# Export cleaned datasets
customers.to_csv("customers_clean.csv", index=False)
cards.to_csv("cards_clean.csv", index=False)
merchants.to_csv("merchants_clean.csv", index=False)
transactions.to_csv("transactions_clean.csv", index=False)
```

## Conclusion

This project demonstrates an end-to-end analytics workflow: raw-data validation and cleaning in Python, business analysis in SQL, and interactive storytelling in Power BI. The strongest portfolio value comes from connecting each KPI or visual to a business question, explaining the insight, and proposing an action.